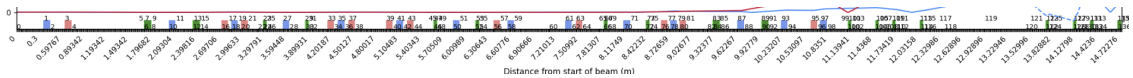
beamline_optimization

April 3, 2026

1 Beamline Optimization - Initialization

This notebook performs sequential optimization of the UH beamline using custom modules and beam parameters.

Help for associated beamline element indexing: image.png



[1]:
```
import os, sys
from pprint import pprint
# pprint(sys.path)

current_dir = os.path.abspath(os.getcwd())

project_root = os.path.abspath(os.path.join(current_dir, '..'))

if project_root not in sys.path:
    sys.path.insert(0, project_root)
pprint(sys.path)
# from backend.beamline import *
```

```
['C:\\Users\\yi_lu\\my_files\\research\\FELsim\\backend_orig',
 'C:\\Users\\yi_lu\\programming_lang\\Anaconda\\envs\\felsim\\python311.zip',
 'C:\\Users\\yi_lu\\programming_lang\\Anaconda\\envs\\felsim\\DLLs',
 'C:\\Users\\yi_lu\\programming_lang\\Anaconda\\envs\\felsim\\Lib',
 'C:\\Users\\yi_lu\\programming_lang\\Anaconda\\envs\\felsim',
 '',
 'C:\\Users\\yi_lu\\AppData\\Roaming\\Python\\Python311\\site-packages',
 'C:\\Users\\yi_lu\\programming_lang\\Anaconda\\envs\\felsim\\Lib\\site-
packages']
```

[2]:
```
# Import Python modules
%matplotlib ipympl
import sys
import time
from pathlib import Path
import numpy as np
```

```python
import pandas as pd
import sympy as sp
import sympy.plotting as plot

# Import Beam Dynamics modules
from ebeam import beam
from beamline import lattice, Beamline
from schematic import draw_beamline
from excelElements import ExcelElements
from beamOptimizer import beamOptimizer
from AlgebraicOptimization import AlgebraicOpti
```

[3]:
```python
# Initial Beam Parameters
Energy = 40  # MeV
f = 2856e6  # Hz
bunch_spread = 2  # ps
energy_std_percent = 0.5  # 0.3 % Energy spread from M. Hadmack Rev. Sci.␣
↪Instrum. 84, 063302 (2013); doi: 10.1063/1.4809938
h = 5e9  # 1/s

epsilon_n = 8  # pi.mm.mrad
x_std = 0.8  # mm
y_std = 0.8  # mm
nb_particles = 1000

relat = lattice(1,fringeType=None)
relat.setE(E=Energy)
norm = relat.gamma * relat.beta
epsilon = epsilon_n / norm
print(relat.gamma)
print(relat.beta)
print(epsilon)
x_prime_std = epsilon / x_std
y_prime_std = epsilon / y_std

tof_std = bunch_spread * 1e-9 * f
gen_tof = np.random.normal(0, tof_std, size=(nb_particles, 1))
energy_std = energy_std_percent * 10

ebeam = beam()
beam_dist = ebeam.gen_6d_gaussian(0,␣
↪[x_std,x_prime_std,y_std,y_prime_std,tof_std,energy_std], nb_particles)
tof_dist = beam_dist[:,4] / f
beam_dist[:,5] += h * tof_dist


# MkV FEL Undulator matching
```

```
K = 1.2
lambda_u = 2.3  # Undulator period in cm

beta_ym = relat.gamma / (K * (2 * np.pi / (lambda_u * 1e-2)))
print(beta_ym)
y_std_m = np.sqrt(epsilon * 1e-6 * beta_ym)
print(y_std_m)
y_prime_std_m = np.sqrt(epsilon * 1e-6 / beta_ym) # Beam at waist so Gamma(z_w)␣
↪= 1 / Beta(z_w)
print(y_prime_std_m)
```

```
79.2794183839214
0.9999204452035254
0.10091694227956337
0.2418394669906551
0.00015622323620773033
0.0006459790792284825
```

[4]:
```
# Load Beamline from Excel
current_path = Path.cwd()


file_path = current_path.parents[1] / 'beam_excel' / 'Beamline_elements.xlsx'



excel = ExcelElements(file_path)
df = excel.get_dataframe()
beamlineUH = excel.create_beamline()
schem = draw_beamline()
line_UH = relat.changeBeamType("electron", Energy, beamlineUH)

# Optimizer and beamline truncation
print('Number of elements in beamline: ' + str(len(line_UH)))
segments = 138
line = line_UH[:segments]
opti = beamOptimizer(line, beam_dist)
```

```
Number of elements in beamline: 138
```

[5]:
```
# Optimization - First Quadrupole Doublet

variables = {
    1: ["I", "current", lambda num:num],
    3: ["I2", "current", lambda num:num],
}
startPoint = {
    "I": {"bounds": (0,10), "start": 1},
```

```
    "I2": {"bounds": (0, 10), "start": 1},
}

objectives = {8: [{"measure": ["x", "alpha"], "goal": 0, "weight":1},
                  {"measure": ["x", "beta"], "goal": 0.1, "weight":0.0}],
              9: [{"measure": ["y", "alpha"], "goal": 0, "weight":1},
                  {"measure": ["y", "beta"], "goal": 0.1, "weight":0.5}]}

result = opti.calc("Nelder-Mead", variables, startPoint, objectives,␣
 ↪plotBeam=False, printResults=True, plotProgress=False)

# old values
# line[1].current = 0.9989681933
# line[3].current = 1.044851479
```

```
x variables:
indice 1 new current value: 0.8294588405236027
indice 3 new current value: 1.045373695861644

y objectives:
indice 8: x alpha value of -0.0012416355113585534
indice 8: x beta value of 1.0364133771427846
indice 9: y alpha value of -0.0007990517118580845
indice 9: y beta value of 0.14591415657621115
Final difference: 0.00026405875735842664

Total time: 121.43041310002445 s
Total iterations: 70
```

[6]:
```
# Optimization - First Chromacity Quad

variables = {10: ["I", "current", lambda num:num]}
startPoint = {"I": {"bounds": (0,10), "start": 1}}
objectives = {15: [{"measure": ["x", "dispersion"], "goal": 0, "weight": 1}]}
result = opti.calc("Nelder-Mead", variables, startPoint, objectives,␣
 ↪plotBeam=False, printResults=True, plotProgress=False)
```

```
x variables:
indice 10 new current value: 3.5398437500000046

y objectives:
indice 15: x disper value of -7.970960914251532e-07
Final difference: 6.9274246797810075e-15

Total time: 63.13323360000504 s
```

```
Total iterations: 38
```

[7]:
```
# Optimization - Quadrupole Triplet

truncated_line = line[:17]
# Add half a quad
#truncated_line.append(qpfLattice(current = 3.816604, length=0.0889/2))

variables = {
    16: ["I", "current", lambda num:num],
    18: ["I2", "current", lambda num:num],
    20: ["I3", "current", lambda num:num],
}
startPoint = {
    "I": {"bounds": (0,10), "start": 2},
    "I2": {"bounds": (0, 10), "start": 5},
    "I3": {"bounds": (0, 10), "start": 3},
}

objectives = {25: [{"measure": ["x", "alpha"], "goal": 0, "weight":1},
                   {"measure": ["x", "beta"], "goal": 0.1, "weight":0.5}],
              26: [{"measure": ["y", "alpha"], "goal": 0, "weight":1},
                   {"measure": ["y", "beta"], "goal": 0.1, "weight":0.5}]}

result = opti.calc("Nelder-Mead", variables, startPoint, objectives,␣
 ↪plotBeam=False, printResults=True, plotProgress=False)

# old values
# line[16].current = 2.4
# line[18].current = 5.108214683
# line[20].current = 3.142089844
```

```
x variables:
indice 16 new current value: 2.13060535165996
indice 18 new current value: 4.931989070008445
indice 20 new current value: 3.482785495895402

y objectives:
indice 25: x alpha value of -0.0004461980633094131
indice 25: x beta value of 0.015795083676805724
indice 26: y alpha value of 0.0002704311105804809
indice 26: y beta value of 0.013469571435413384
Final difference: 0.0018223159314952149

Total time: 167.50390320000588 s
Total iterations: 97
```

[8]:
```
# Optimization - Second Chromacity Quad

variables = {27: ["I", "current", lambda num:num]}
startPoint = {"I": {"bounds": (0,10), "start": 1}}
objectives = {32: [{"measure": ["x", "dispersion"], "goal": 0, "weight": 1}]}
result = opti.calc("Nelder-Mead", variables, startPoint, objectives,␣
 ↪plotBeam=False, printResults=True, plotProgress=False)
```

```
x variables:
indice 27 new current value: 4.510156250000008

y objectives:
indice 32: x disper value of -1.2504845566351462e-06
Final difference: 2.2786944205569025e-14

Total time: 73.89435450002202 s
Total iterations: 40
```

[9]:
```
# Optimization - Double Quadrupole Triplet

variables = {
    37: ["I", "current", lambda num:num],
    35: ["I2", "current", lambda num:num],
    33: ["I3", "current", lambda num:num],
}
startPoint = {
    "I": {"bounds": (0,10), "start": 2},
    "I2": {"bounds": (0, 10), "start": 2},
    "I3": {"bounds": (0, 10), "start": 2},
}
objectives = {
    37: [
        {"measure": ["x", "alpha"], "goal": 0, "weight": 1},
        {"measure": ["y", "alpha"], "goal": 0, "weight": 1},
        {"measure": ["x", "envelope"], "goal": 2.0, "weight": 1},
        {"measure": ["y", "envelope"], "goal": 2.0, "weight": 1}
    ]
}
result = opti.calc("Nelder-Mead", variables, startPoint, objectives,␣
 ↪plotBeam=False, printResults=True, plotProgress=False)

line[43].current = line[33].current
line[41].current = line[35].current
line[39].current = line[37].current
```

```
x variables:
indice 37 new current value: 0.0
indice 35 new current value: 2.3832471810606997
indice 33 new current value: 2.6879996970905555

y objectives:
indice 37: x alpha value of -0.016911515197462316
indice 37: y alpha value of -0.008604170247908315
indice 37: x envelope value of 1.2041182917256514
indice 37: y envelope value of 1.413165347858333
Final difference: 0.24454059968186956

Total time: 453.83090329999686 s
Total iterations: 296
```

[10]:
```
# Optimization - Third Chromacity Quad

variables = {50: ["I", "current", lambda num:num]}
startPoint = {"I": {"bounds": (0,10), "start": 1}}
objectives = {55: [{"measure": ["x", "dispersion"], "goal": 0, "weight": 1}]}
result = opti.calc("Nelder-Mead", variables, startPoint, objectives,␣
 ↪plotBeam=False, printResults=True, plotProgress=False)
```

```
x variables:
indice 50 new current value: 4.615917968750008

y objectives:
indice 55: x disper value of 4.348793295676319e-08
Final difference: 1.89120031285193e-15

Total time: 56.36721409999882 s
Total iterations: 40
```

[11]:
```
# Optimization - Quadrupole Doublet and Interaction Point (z = 7.11 m, end of␣
 ↪element index = 59)

variables = {
    56: ["I", "current", lambda num:num],
    58: ["I2", "current", lambda num:num],
}
startPoint = {
    "I": {"bounds": (0,10), "start": 2},
    "I2": {"bounds": (0, 10), "start": 2},
}
```

```
objectives = {
    59: [
        {"measure": ["x", "envelope"], "goal": 0.0, "weight": 1},
        {"measure": ["y", "envelope"], "goal": 0.0, "weight": 1}
    ]
}

result = opti.calc("Nelder-Mead", variables, startPoint, objectives,␣
 ↪plotBeam=False, printResults=True, plotProgress=False)
```

```
x variables:
indice 56 new current value: 3.1212305222954377
indice 58 new current value: 3.3065637789219093

y objectives:
indice 59: x envelope value of 0.022257801211600398
indice 59: y envelope value of 0.05501813124913204
Final difference: 0.0017612021155694246

Total time: 112.86168960001669 s
Total iterations: 78
```

[12]:
```
# Optimization - Quadrupole Doublet

variables = {
    61: ["I", "current", lambda num:num],
    63: ["I2", "current", lambda num:num],
}
startPoint = {
    "I": {"bounds": (0,10), "start": 2},
    "I2": {"bounds": (0, 10), "start": 2},
}

objectives = {
    68: [{"measure": ["x", "alpha"], "goal": 0, "weight": 1},
        {"measure": ["x", "beta"], "goal": 0.1, "weight": 0.5}],
    69: [{"measure": ["y", "alpha"], "goal": 0, "weight": 1},
         {"measure": ["y", "beta"], "goal": 0.1, "weight": 0.5}]
}

result = opti.calc("Nelder-Mead", variables, startPoint, objectives,␣
 ↪plotBeam=False, printResults=True, plotProgress=False)
```

```
x variables:
```

```
indice 61 new current value: 5.1732924805612654
indice 63 new current value: 4.042245023645947

y objectives:
indice 68: x alpha value of 0.0006261455924151993
indice 68: x beta value of 0.06269994890480632
indice 69: y alpha value of 0.000654862945433761
indice 69: y beta value of 0.01679165027967871
Final difference: 0.0010395706353054698

Total time: 129.19784950002213 s
Total iterations: 99
```

[13]:
```
# Optimization - Fourth Chromacity Quad

variables = {70: ["I", "current", lambda num:num]}
startPoint = {"I": {"bounds": (0,10), "start": 1}}
objectives = {75: [{"measure": ["x", "dispersion"], "goal": 0, "weight": 1}]}
result = opti.calc("Nelder-Mead", variables, startPoint, objectives,␣
 ↪plotBeam=False, printResults=True, plotProgress=False)
```

```
x variables:
indice 70 new current value: 4.757031250000009

y objectives:
indice 75: x disper value of 6.57375932842412e-07
Final difference: 1.5286128253785284e-13

Total time: 51.08909009999479 s
Total iterations: 40
```

[14]:
```
# Optimization - Quadrupole Triplet

variables = {
    76: ["I", "current", lambda num:num],
    78: ["I2", "current", lambda num:num],
    80: ["I3", "current", lambda num:num],
}
startPoint = {
    "I": {"bounds": (0,10), "start": 2},
    "I2": {"bounds": (0, 10), "start": 2},
    "I3": {"bounds": (0, 10), "start": 2},
}

objectives = {
```

```
    85: [{"measure": ["x", "alpha"], "goal": 0, "weight": 1},
        {"measure": ["x", "beta"], "goal": 0.1, "weight": 0.5}],
    86: [{"measure": ["y", "alpha"], "goal": 0, "weight": 1},
         {"measure": ["y", "beta"], "goal": 0.1, "weight": 0.5}]
}

result = opti.calc("Nelder-Mead", variables, startPoint, objectives,␣
 ↪plotBeam=False, printResults=True, plotProgress=False)
```

```
x variables:
indice 76 new current value: 3.9150166957178576
indice 78 new current value: 4.013976475444643
indice 80 new current value: 0.0

y objectives:
indice 85: x alpha value of 0.000512467827508933
indice 85: x beta value of 0.07077263098337801
indice 86: y alpha value of 0.0003637910325527264
indice 86: y beta value of 0.09994477795207041
Final difference: 0.00010687901043594602

Total time: 202.37342630000785 s
Total iterations: 129
```

[15]:
```
# Optimization - Fifth Chromacity Quad

variables = {87: ["I", "current", lambda num:num]}
startPoint = {"I": {"bounds": (0,10), "start": 1}}
objectives = {92: [{"measure": ["x", "dispersion"], "goal": 0, "weight": 1}]}
result = opti.calc("Nelder-Mead", variables, startPoint, objectives,␣
 ↪plotBeam=False, printResults=True, plotProgress=False)
```

```
x variables:
indice 87 new current value: 4.053027343750006

y objectives:
indice 92: x disper value of 3.415791191173665e-08
Final difference: 1.1667629461699605e-15

Total time: 67.56455019998248 s
Total iterations: 40
```

[16]:
```
# Optimization - Quadrupole Triplet and MkIII undulator start (z = 12.389 m ,␣
 ↪end of element index = 117)
```

```
variables = {
    93: ["I", "current", lambda num:num],
    95: ["I2", "current", lambda num:num],
    97: ["I3", "current", lambda num:num],
}
startPoint = {
    "I": {"bounds": (0,10), "start": 2},
    "I2": {"bounds": (0, 10), "start": 2},
    "I3": {"bounds": (0, 10), "start": 2},
}
objectives = {
    117: [
        {"measure": ["x", "alpha"], "goal": 0, "weight": 1},
        {"measure": ["y", "alpha"], "goal": 0, "weight": 1},
        {"measure": ["x", "beta"], "goal": beta_ym, "weight": 1},
        {"measure": ["y", "beta"], "goal": beta_ym, "weight": 1}
    ]
}
result = opti.calc("Nelder-Mead", variables, startPoint, objectives,␣
 ↪plotBeam=False, printResults=True, plotProgress=False)


#indice 93 new current value: 2.1275944045328767
#indice 95 new current value: 1.543397467601471
#indice 97 new current value: 0.37986577370764 52
```

```
x variables:
indice 93 new current value: 2.0315541304722594e-06
indice 95 new current value: 3.3409896025903247
indice 97 new current value: 2.57888040133256

y objectives:
indice 117: x alpha value of 0.02416019696853613
indice 117: y alpha value of -2.5766728519662847e-06
indice 117: x beta value of 0.21068143050430055
indice 117: y beta value of 0.03608718422324383
Final difference: 0.01097211443933393

Total time: 576.8788305999769 s
Total iterations: 357
```

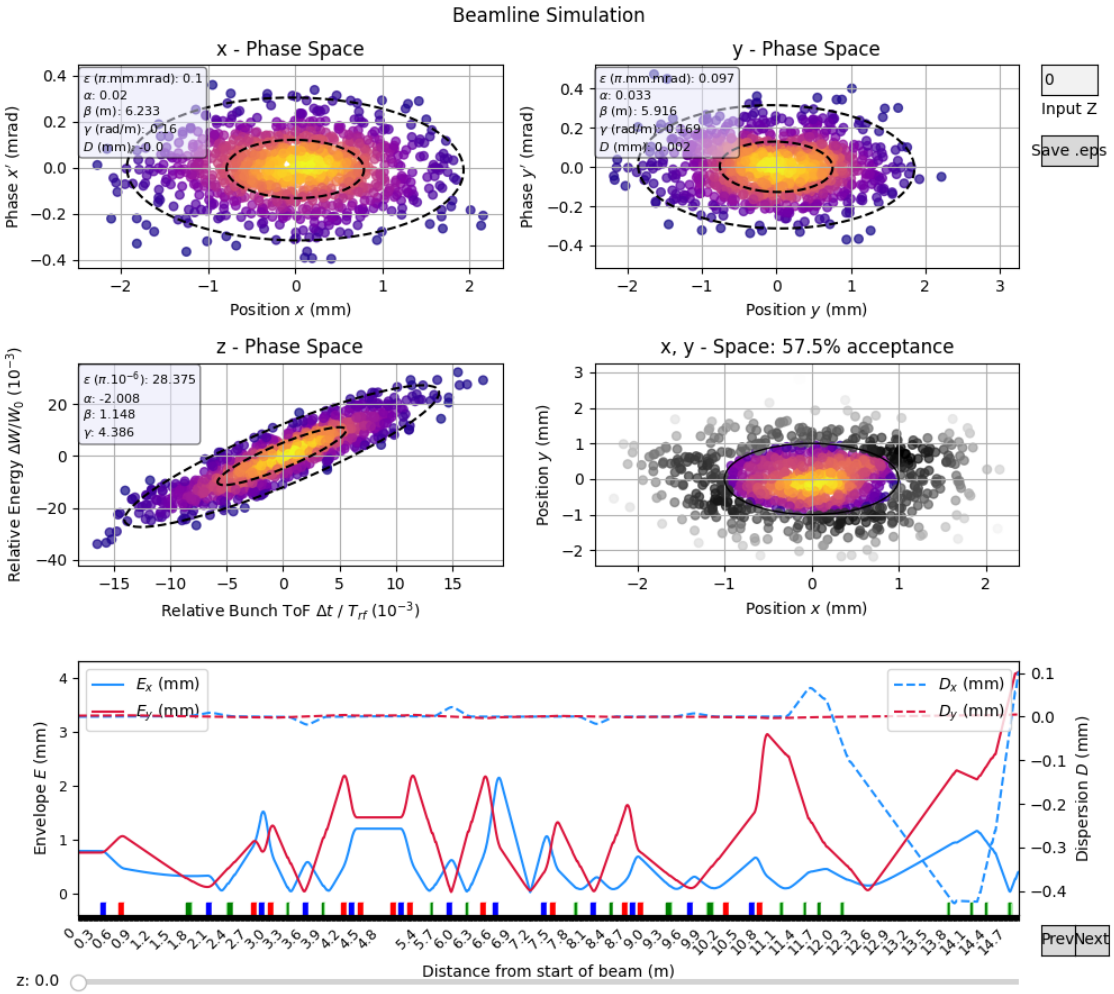
[17]:
```
# Display Optimized Beamline

%matplotlib ipympl
```

```
acceptance = {"shape":'circle', "radius":1, "origin":[0,0]}
schem.plotBeamPositionTransform(beam_dist, line, 0.01, plot=True,␣
↪showIndice=False,
                                defineLim=False, matchScaling=False,␣
↪shape=acceptance, scatter=True)
```

```
Simulating Beamline:   0%|
| 0/1550 [00:00<?]

Simulating Beamline:   3%|
| 48/1550 [00:00<00:03]

Simulating Beamline:   6%|
| 96/1550 [00:00<00:04]

Simulating Beamline:   9%|
| 143/1550 [00:00<00:03]

Simulating Beamline:  12%|
| 183/1550 [00:00<00:04]

Simulating Beamline:  15%|
| 232/1550 [00:00<00:03]

Simulating Beamline:  18%|
| 272/1550 [00:00<00:04]

Simulating Beamline:  21%|
| 318/1550 [00:00<00:03]

Simulating Beamline:  23%|
| 356/1550 [00:01<00:04]

Simulating Beamline:  25%|
| 393/1550 [00:01<00:04]

Simulating Beamline:  27%|
| 424/1550 [00:01<00:04]

Simulating Beamline:  30%|
| 472/1550 [00:01<00:03]

Simulating Beamline:  33%|
| 512/1550 [00:01<00:03]

Simulating Beamline:  36%|
| 553/1550 [00:01<00:02]

Simulating Beamline:  38%|
| 591/1550 [00:01<00:03]

Simulating Beamline:  41%|
| 630/1550 [00:01<00:03]

Simulating Beamline:  43%|
| 663/1550 [00:02<00:03]
```

```
 Simulating Beamline:  46%|
| 710/1550 [00:02<00:02]

 Simulating Beamline:  49%|
| 761/1550 [00:02<00:02]

 Simulating Beamline:  52%|
| 806/1550 [00:02<00:02]

 Simulating Beamline:  54%|
| 841/1550 [00:02<00:02]

 Simulating Beamline:  56%|
| 874/1550 [00:02<00:02]

 Simulating Beamline:  59%|
| 921/1550 [00:02<00:01]

 Simulating Beamline:  62%|
| 957/1550 [00:03<00:01]

 Simulating Beamline:  64%|
| 989/1550 [00:03<00:01]

 Simulating Beamline:  66%|
| 1023/1550 [00:03<00:01]

 Simulating Beamline:
68%|
| 1058/1550 [00:03<00:01]

 Simulating Beamline:
71%|
| 1101/1550 [00:03<00:01]

 Simulating Beamline:
73%|
| 1136/1550 [00:03<00:01]

 Simulating Beamline:
75%|
| 1170/1550 [00:03<00:01]

 Simulating Beamline:
77%|
| 1199/1550 [00:04<00:01]

 Simulating Beamline:
79%|
| 1224/1550 [00:04<00:01]

 Simulating Beamline:
80%|
| 1246/1550 [00:04<00:01]
```

```
 Simulating Beamline:
82%|
| 1278/1550 [00:04<00:01]

 Simulating Beamline:
84%|
| 1309/1550 [00:04<00:01]

 Simulating Beamline:
86%|
| 1338/1550 [00:04<00:00]

 Simulating Beamline:
89%|
| 1373/1550 [00:04<00:00]

 Simulating Beamline:
91%|
| 1406/1550 [00:04<00:00]

 Simulating Beamline:
92%|
| 1432/1550 [00:05<00:00]

 Simulating Beamline:
94%|
| 1456/1550 [00:05<00:00]

 Simulating Beamline:
95%|
| 1478/1550 [00:05<00:00]

 Simulating Beamline:  97%|
               | 1502/1550 [00:05<00:00]

 Simulating Beamline:  98%|
               | 1514/1550 [00:05<00:00]
```



[17]:
```
                                   $\epsilon$ ($\pi$.mm.mrad)  \
x  [0.09984433665487545, 0.09984433665487548, 0.0…
y  [0.09728062352064012, 0.09728062352064015, 0.0…
z  [28.375105586767294, 28.375105586767287, 28.37…

                                                $\alpha$  \
x  [0.01981756049586989, 0.01821268122977514, 0.0…
y  [0.032540092928050254, 0.030847851205950832, 0…
z  [-2.008186972627923, -2.0081213173605277, -2.0…

                                             $\beta$ (m)  \
x  [6.2334454487553455, 6.233065146338088, 6.2327…
y  [5.915578398609918, 5.9149445191685786, 5.9143…
z  [1.1475290665393274, 1.147468942333681, 1.1474…

                                          $\gamma$ (rad/m)  \
```

```
x  [0.160487926609474, 0.16048792660947397, 0.160…
y  [0.16922417220994004, 0.16922417220994, 0.1692…
z  [4.385784259226013, 4.385784259226014, 4.38578…

                                            $D$ (mm)  \
x  [-0.00022700663007559302, -0.00022119791405981…
y  [0.002163280453316047, 0.002163591206628182, 0…
z  [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, …

                                 $D^{\prime}$ (mrad)  \
x  [0.0005808716015773609, 0.0005808716015773609,…
y  [3.1075331213283454e-05, 3.1075331213283454e-0…
z  [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, …

                                          $\phi$ (deg)  \
x  [89.81303237323712, 89.82816237399369, 89.8432…
y  [89.67556293804277, 89.6923999687794, 89.70924…
z  [-25.561019701288227, -25.56030213960558, -25…

                                   Envelope $E$ (mm)
x  [0.7889069817825989, 0.7888829158137161, 0.788…
y  [0.7585981512645563, 0.7585575066629506, 0.758…
z  [5.706247315612769, 5.706097825682134, 5.70594…
```

[ ]:

```
```